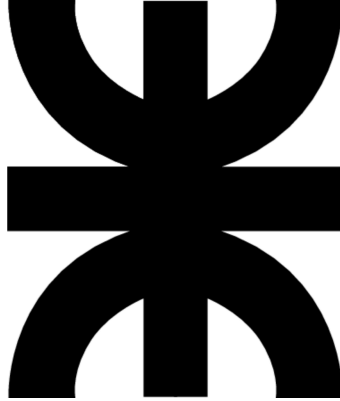


Proyecto Smart Home



Integrantes:

- Fernandez, Orianna Del Rosario
- Aratto, José Alejandro
- Cerqueiro, Carabajal Octavio
- Taus, Fernando Andrés
- Hochmuth, Nehueln Lautaro
- Goy, Bruno Sebastian

Asignatura: Sintaxis y Semántica de los Lenguajes

Carrera: Ing. en Sistemas de Información

Primer cuatrimestre

Curso académico: 2026

Universidad y regional: UTN FRRe

Lugar y fecha: Resistencia, 2026

Índice

1. Matriz de habilidades.....	2
2. Identidad.....	2
3. Alianza de equipo.....	3
4. Gramática.....	3
4.1 Símbolos no terminales.....	3
4.2 Símbolos terminales.....	4
4.3 Reglas de producción.....	4
4.3.1 Regla inicial.....	4
4.3.2 Lista de sentencias.....	4
4.3.3 Tipos de sentencia.....	4
4.3.4 Estructura del bloque WHEN.....	5
4.3.5 Estructura del bloque EVERY.....	5
4.3.6 Estructura del bloque IF.....	5
4.3.7 Reglas de asignación.....	5
4.3.8 Estructura de bloque.....	6
4.3.9 Regla general de condición.....	6
4.3.10 Expresiones lógicas OR.....	6
4.3.11 Expresiones lógicas AND.....	6
4.3.12 Expresiones lógicas NOT.....	6
4.3.13 Condiciones primarias.....	6
4.3.14 Reglas de comparación.....	6
4.3.15 Expresiones temporales.....	8
4.3.16 Operadores.....	8
5. Lexer.....	8
5.1 Introducción.....	8
5.1.1 Propósito del lenguaje.....	8
5.1.2 Archivos del proyecto.....	8
5.2. Configuración del Lexer.....	8
5.2.1 Directivas Flex.....	8
5.2.2 Configuración de C.....	9
5.3 Manejo de Errores Léxicos.....	9

1. Matriz de habilidades

Integrantes	Inglés	Edición de video	Habilidades de Programación	Búsqueda de información	Documentación	Idear soluciones
Aratto José Alejandro	○	★	★	○		★
Taus Fernando Andrés	★	★	★	○	○	★
Hochmuth Nehueln Lautaro	★	○	★	★		○
Goy Bruno Sebastian	★		○	★	○	★
Cerqueiro Carabajal Octavio	★	○	○	★	★	★
Fernandez Orianna Del Rosario	★	★	★		○	○

2. Identidad

Nombre: C.mánticos

Logo:



3. Alianza de equipo

Como grupo, acordamos organizarnos de manera colaborativa y responsable para el desarrollo del trabajo práctico. Nuestro objetivo principal es lograr un proyecto sólido, bien documentado y funcional, distribuyendo las tareas de forma equitativa y aprovechando las fortalezas individuales de cada integrante. Nos proponemos que al menos dos personas del equipo puedan desenvolverse con fluidez en tareas claves como la programación, documentación y planificación del proyecto.

Nos comprometemos a mantener reuniones de trabajo y seguimiento al menos tres veces por semana, priorizando los encuentros nocturnos durante la semana y en la mañana los fines de semana. Estas reuniones se realizan de manera virtual a través de Discord, y utilizaremos GitHub como plataforma principal para el seguimiento del proyecto y mantenerlo siempre actualizado.

Nos comunicaremos de forma constante y fluida por Discord, y en caso de ser necesario, también por otros canales complementarios como WhatsApp.

Todos los integrantes del equipo nos comprometemos a trabajar de forma constante a lo largo del plazo establecido, evitando dejar tareas para último momento, con el objetivo de mantener una buena calidad del trabajo y reducir el estrés asociado a la entrega.

Las metas y valores comunes en nuestro equipo son:

- Responsabilidad individual y grupal.
- Buena comunicación continua.
- Organización y planificación para cumplir en tiempo y forma las entregas.
- Apoyo en dificultades generales.
- Abierto a escuchar opiniones distintas a lo largo del desarrollo y debatir soluciones.

4. Gramática

4.1 Símbolos no terminales

programa, lista_sentencias, sentencia, bloque, when, every, if, asignacion, asignacion_foco, asignacion_aire, asignacion_persiana, asignacion_cerradura, asignacion_altavoz, asignacion_alarma, modo_aire, condicion, expresion_or, expresion_and, expresion_not, primaria_condicion, comparacion, comparacion_temp,

comparacion_hum, comparacion_luz, comparacion_mov, comparacion_humo,
comparacion_reloj, comparacion_actuador, tiempo

4.2 Símbolos terminales

Palabras reservadas

WHEN, IF, THEN, ELSE, DO, END, EVERY, AND, OR, NOT

Atributos y Modos

estado, brillo, color, modo, temp_obj, temp_objetivo, temp_act, posicion, volumen,
mute, mensaje, email_notif, email, activada, hora, fecha, FRIO, CALOR, VENT

Operadores

==, !=, >, <, >=, <=, =, .

Booleanos

TRUE, FALSE, ON, OFF

Unidades

°C, %, lux, s, m, h

Identificadores y literales

FOCO_ID, AIRE_ID, PERSIANA_ID, CERRADURA_ID, ALTAVOZ_ID, ALARMA_ID,
SENSOR_TEMPERATURA_ID, SENSOR_HUMEDAD_ID, SENSOR_LUZ_ID,
SENSOR_MOVIMIENTO_ID, SENSOR_HUMO_ID, RELOJ_ID, BOOLEANO,
PORCENTAJE, TEMPERATURA, ILUMINANCIA, TIEMPO, TEXTO, VALOR_HORA,
VALOR_FECHA, EMAIL

4.3 Reglas de producción

4.3.1 Regla inicial

programa → lista_sentencias

4.3.2 Lista de sentencias

lista_sentencias → sentencia lista_sentencias

lista_sentencias → sentencia

4.3.3 Tipos de sentencia

sentencia → when

sentencia → every

sentencia → if

sentencia → asignacion

4.3.4 Estructura del bloque WHEN

when → WHEN condicion DO bloque END

4.3.5 Estructura del bloque EVERY

every → EVERY tiempo DO bloque END

4.3.6 Estructura del bloque IF

if → IF condicion THEN bloque ELSE bloque END

if → IF condicion THEN bloque END

4.3.7 Reglas de asignación

asignacion → asignacion_foco

asignacion → asignacion_aire

asignacion → asignacion_persiana

asignacion → asignacion_cerradura

asignacion → asignacion_altavoz

asignacion → asignacion_alarma

asignacion_foco → FOCO_ID . color = VALOR_COLOR

VALOR_COLOR → blanco | rojo | azul

asignacion_foco → FOCO_ID . estado = BOOLEANO

asignacion_foco → FOCO_ID . brillo = PORCENTAJE

asignacion_aire → AIRE_ID . temp_obj = TEMPERATURA

asignacion_aire → AIRE_ID . temp_objetivo = TEMPERATURA

asignacion_aire → AIRE_ID . estado = BOOLEANO

asignacion_aire → AIRE_ID . modo = modo_aire

modo_aire → FRIO | CALOR | VENT

asignacion_persiana → PERSIANA_ID . posicion = PORCENTAJE

asignacion_cerradura → CERRADURA_ID . estado = BOOLEANO

asignacion_altavoz → ALTAVOZ_ID . volumen = PORCENTAJE

asignacion_altavoz → ALTAVOZ_ID . mute = BOOLEANO

asignacion_altavoz → ALTAVOZ_ID . mensaje = TEXTO

asignacion_altavoz → ALTAVOZ_ID . email_notif = EMAIL

asignacion_altavoz → ALTAVOZ_ID . email = EMAIL

asignacion_alarma → ALARMA_ID . estado = BOOLEANO

asignacion_alarma → ALARMA_ID . activada = BOOLEANO

4.3.8 Estructura de bloque

bloque → sentencia bloque

bloque → sentencia

4.3.9 Regla general de condición

condicion → expresion_or

4.3.10 Expresiones lógicas OR

expresion_or → expresion_and OR expresion_or

expresion_or → expresion_and

4.3.11 Expresiones lógicas AND

expresion_and → expresion_not AND expresion_and

expresion_and → expresion_not

4.3.12 Expresiones lógicas NOT

expresion_not → NOT expresion_not

expresion_not → primaria_condicion

4.3.13 Condiciones primarias

primaria_condicion → comparacion

primaria_condicion → BOOLEANO

4.3.14 Reglas de comparación

comparacion → comparacion_temp

comparacion → comparacion_hum

comparacion → comparacion_luz

comparacion → comparacion_mov

comparacion → comparacion_humo

comparacion → comparacion_reloj

comparacion → comparacion_actuador

comparacion_temp → SENSOR_TEMPERATURA_ID OPERADOR_COMP
TEMPERATURA

comparacion_hum → SENSOR_HUMEDAD_ID OPERADOR_COMP PORCENTAJE

comparacion_luz → SENSOR_LUZ_ID OPERADOR_COMP ILUMINANCIA

comparacion_mov → SENSOR_MOVIMIENTO_ID OP_IGUALDAD BOOLEANO

comparacion_humo → SENSOR_HUMO_ID OP_IGUALDAD BOOLEANO

comparacion_reloj → RELOJ_ID . hora OPERADOR_COMP VALOR_HORA

comparacion_reloj → RELOJ_ID . fecha OPERADOR_COMP VALOR_FECHA

comparacion_actuador → FOCO_ID . estado OP_IGUALDAD BOOLEANO

comparacion_actuador → FOCO_ID . brillo OPERADOR_COMP PORCENTAJE

comparacion_actuador → FOCO_ID . color OP_IGUALDAD VALOR_COLOR

comparacion_actuador → AIRE_ID . estado OP_IGUALDAD BOOLEANO

comparacion_actuador → AIRE_ID . modo OP_IGUALDAD modo_aire

comparacion_actuador → AIRE_ID . temp_obj OPERADOR_COMP TEMPERATURA

comparacion_actuador → AIRE_ID . temp_act OPERADOR_COMP TEMPERATURA

comparacion_actuador → PERSIANA_ID . posicion OPERADOR_COMP
PORCENTAJE

comparacion_actuador → CERRADURA_ID . estado OP_IGUALDAD BOOLEANO

comparacion_actuador → ALTAVOZ_ID . volumen OPERADOR_COMP
PORCENTAJE

comparacion_actuador → ALTAVOZ_ID . mute OP_IGUALDAD BOOLEANO

comparacion_actuador → ALARMA_ID . estado OP_IGUALDAD BOOLEANO

comparacion_actuador → ALARMA_ID . activada OP_IGUALDAD BOOLEANO

4.3.15 Expresiones temporales

tiempo → TIEMPO

tiempo → VALOR_HORA

4.3.16 Operadores

OP_IGUALDAD → == | !=

OP_RELACIONAL → > | < | >= | <=

OPERADOR_COMP → OP_IGUALDAD | OP_RELACIONAL

5. Lexer

5.1 Introducción

El analizador léxico (lexer) es el componente encargado de transformar una secuencia de caracteres de entrada en una secuencia de tokens significativos para el lenguaje **SmartHome DSL**. Está implementado utilizando la herramienta Flex (específicamente win_flex para Windows) y genera el archivo C **lex.yy.c** que contiene el autómata finito determinista (DFD) utilizado para el reconocimiento.

5.1.1 Propósito del lenguaje

SmartHome DSL es un lenguaje de dominio específico para la programación de hogares inteligentes. Permite definir reglas automatizadas basadas en sensores y condiciones para controlar actuadores como luces, persianas, alarmas, aire acondicionado, cerraduras y altavoces.

5.1.2 Archivos del proyecto

- src/Lexer.l -- Archivo fuente del lexer (definiciones Flex).
- src/lex.yy.c -- Código C generado por Flex.
- bin/lexer.exe -- Ejecutable compilado.

5.2. Configuración del Lexer

Las opciones de configuración se definen al inicio del archivo Lexer.l:

5.2.1 Directivas Flex

- %option noyywrap
- %option case-insensitive
- %option 8bit

- **noyywrap**: Indica que no se utilizará la función yywrap(). Al llegar al final del archivo, el lexer termina sin buscar más entrada.
- **case-insensitive**: Hace que todos los patrones sean insensibles a mayúsculas/minúsculas. Así, WHEN, when y When son reconocidos como el mismo token.
- **8bit**: Permite el manejo de caracteres de 8 bits, necesario para el soporte de UTF-8 y caracteres extendidos como el símbolo de grado.

5.2.2 Configuración de C

El código C embebido configura la consola para usar UTF-8 y establece la locale del sistema:

- SetConsoleOutputCP(65001);
- SetConsoleCP(65001);
- setlocale(LC_ALL, "");

5.3 Manejo de Errores Léxicos

El lexer implementa una regla de error unificado al final de la sección de reglas:

```
[a-zA-Z0-9_]+|[xC2-\xDF][x80-\xBF]|. {
    printf("TOKEN_ERROR: %s (En linea %d)\n", yytext, linea);
}
```